



华南理工大学
South China University of Technology

操作系统实验报告

郑重声明：

- 1、实验手册中的所有实验均有本人独立编码、调试和测试。
- 2、实验手册中给出的实验数据和结果完全由本人所完成的程序给出。
- 3、本人了解：不按照前两条要求所完成的实验报告已经构成了抄袭或造假行为，本人将承担相应的不良后果。

姓名：_____（签名）

学号：_____

班级：_____22 计算机科学与技术 1 班

提交日期：_____2025/06/08

成绩：_____

一、实验目的

存储管理的主要功能之一是合理地分配空间。请求页式管理是一种常用的虚拟存储管理技术。

本实验的目的是通过请求页式存储管理中页面置换算法模拟设计，了解虚拟存储技术的技术特点，掌握请求页式存储管理的页面置换算法。

二、实验内容

(1) 通过随机数产生一个指令序列，共 320 条指令。指令的地址按下述原则生成：

- ①50%的指令是顺序执行的；
- ②25%的指令是均匀分布在前地址部分；
- ③25%的指令是均匀分布在后地址部分。

具体的实施方法是：

- ①在 $[0, 319]$ 的指令之间随即选取一起点 m ；
- ②顺序执行一条指令，即执行地址为 $m+1$ 的指令；
- ③在前地址 $[0, m+1]$ 中随机选取一条指令并执行，该指令的地址为 m' ；
- ④顺序执行一条指令，其地址为 $m' + 1$ ；
- ⑤在后地址 $[m' + 2, 319]$ 中随机选取一条指令并执行；
- ⑥重复上述步骤①-⑤，直到执行 320 次指令。

(2) 将指令序列变换为页地址流

设：①页面大小为 $1k$ ；

②用户内存容量为 4 页到 32 页；

③用户虚存容量为 $32k$ 。

在用户虚存中，按每 k 存放 10 条指令排在虚存地址，即 320 条指令在虚存中的存放方式为：

第 0 条-第 9 条指令为第 0 页（对应虚存地址为 $[0, 9]$ ）；

第 10 条-第 19 条指令为第一页（对应虚存地址为 $[10, 19]$ ）；

... ..

第 310 条~第 319 条指令为第 31 页（对应虚地址为 $[310, 319]$ ）。

按以上方式，用户指令可组成 32 页。

(3) 计算并输出下述各种算法在不同内存容量下的命中率。

- ①先进先出的算法（FIFO）；
- ②最近最少使用算法（LRU）；
- ③最佳淘汰算法（OPT），先淘汰最不常用的页地址；
- ④最近未使用算法（NRU）。

命中率 = $1 - \text{页面失效次数} / \text{页地址流长度}$

在本实验中，页地址流长度为 320，页面失效次数为每次访问相应指令时，该指令所对应的页不在内存的次数。

3、随机数产生办法，Linux 或 UNIX 系统提供函数 `strand()` 和 `rand()`，分别进行初始化和产生随机数。例如：

```
srand ();
```

语句可初始化一个随机数；

```
a[0]=10*rand()/65535*319+1;
```

```
a[1]=10*rand()/65535*a[0];
```

语句可用来产生 $a[0]$ 与 $a[1]$ 中的随机数。

三、实验环境

编译标准: C++ 17

四、实验步骤（附上截图）

使用以下数据结构实现页面置换算法:

FIFO: 队列;

LRU: 哈希+双向链表;

OPT: 数组搜索;

NRU: 分级随机列表

页框	FIFO(%)	LRU(%)	OPT(%)	NRU(%)
4	60.94	60.94	70.00	60.62
5	62.19	62.81	72.50	62.81
6	63.12	64.38	74.38	64.06
7	65.00	65.31	76.25	69.69
8	66.25	66.88	77.81	68.44
9	66.56	67.50	79.38	71.56
10	66.88	68.12	80.62	71.88
11	69.69	68.44	81.88	72.19
12	70.94	69.69	82.81	72.81
13	71.88	71.56	83.44	73.75
14	73.12	73.44	84.06	74.69
15	73.44	75.00	84.69	75.62
16	75.00	75.62	85.31	76.88
17	75.31	77.19	85.94	78.75
18	77.81	78.12	86.56	79.38
19	78.44	80.00	87.19	80.00
20	79.69	80.31	87.81	81.88
21	82.50	81.56	88.44	82.19
22	84.38	82.81	88.75	83.75
23	84.38	84.38	89.06	84.06
24	84.69	85.00	89.38	85.00
25	85.62	86.25	89.69	85.00
26	86.25	87.19	90.00	85.94
27	87.50	87.50	90.00	87.19
28	88.12	88.12	90.00	88.44
29	88.12	88.44	90.00	89.06
30	88.75	89.06	90.00	89.38
31	89.69	89.38	90.00	89.38
32	90.00	90.00	90.00	90.00

五、实验结果及分析

OPT 算法显著优于其他算法。LRU 次优, 逼近 OPT。FIFO 表现最差, 且可能出现 Belady 异常 (如下图红框所示)。

页框	FIFO(%)	LRU(%)	OPT(%)	NRU(%)
4	56.56	57.50	66.88	56.56
5	57.50	57.81	69.38	58.44
6	58.75	58.75	71.56	60.62
7	60.62	60.31	73.44	62.50
8	62.81	62.19	75.31	65.94
9	66.56	65.00	77.19	67.50
10	66.88	67.81	78.75	69.06
11	68.75	68.44	80.00	69.69
12	69.38	70.31	81.25	70.62
13	70.94	71.25	82.50	71.88
14	72.19	71.88	83.44	74.38
15	72.81	72.81	84.38	76.88

备注: 将代码和实验报告打包发邮件

```

#include <iostream>
#include <vector>
#include <queue>
#include <list>
#include <unordered_map>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <iomanip>

using namespace std;

vector<int> gen_instruction_sequence(int N_instr = 320) {
    /*生成长度为 N_instr 的指令地址序列
    *参数:
    *N_instr: 指令地址长度, 默认 320 条
    *return:
    *指令地址序列
    */
    vector<int> seq(N_instr);
    srand((unsigned)time(nullptr)); // 使用当前时间作为随机种子

    seq[0] = rand() * (N_instr - 1) / RAND_MAX; // 第一条指令的随机地址
    for (int i = 1; i < N_instr; ++i) {
        double p = rand() / (double)RAND_MAX; // 生成 0~1 之间的概率
        int prev = seq[i-1]; // 上一条指令地址
        if (p < 0.50) {
            // 顺序执行(50% 情况)
            seq[i] = (prev + 1) % N_instr;
        } else if (p < 0.75) {
            // 前区跳转(25% 情况)
            seq[i] = rand() * (prev + 1) / RAND_MAX;
        } else {
            // 后区跳转(25% 情况)
            if (prev + 2 >= N_instr) { // 如果越界, 则在整个区间随机
                seq[i] = rand() % N_instr;
            } else {
                seq[i] = prev + 2 + rand() * (N_instr - (prev + 2)) / RAND_MAX;
            }
        }
    }
    return seq;
}

```

```

vector<int> addr_to_page(const vector<int>& seq, int instr_per_page = 10) {
    /* 将指令地址序列转换为页号序列，每 10 条指令为一页
    *参数：
    *seq: 指令地址的序列
    *instr_per_page: 每一页包含多少条指令
    *return:
    *页号序列
    */

    vector<int> pages;
    pages.reserve(seq.size()); //预留空间提高效率
    for (int addr : seq) {
        pages.push_back(addr / instr_per_page); // 地址除以每页指令数，得到页号
    }
    return pages;
}

```

```

int simulateFIFO(const vector<int>& pages, int capacity) {
    /*
    *FIFO 页面置换模拟
    *参数：
    *pages: 页号序列
    *capacity: 页框数
    *return:
    *缺页次数
    */
    queue<int> q; //队列维护内存中的页面顺序
    unordered_map<int,bool> in_mem; // 记录页面是否在内存中
    int faults = 0;
    for (int p : pages) {
        if (!in_mem[p]) { //页面不在内存中，即缺页
            faults++;
            if ((int)q.size() == capacity) { // 判断内存是否已满，如是弹出队首页面
                int old = q.front(); q.pop();
                in_mem[old] = false;
            }
            q.push(p); // 将当前页面加入队尾
            in_mem[p] = true;
        }
    }
    return faults;
}

```

```

int simulateLRU(const vector<int>& pages, int capacity) {

```

```

/*
 *LRU 页面置换模拟
 *参数:
 *pages:页号序列
 *capacity:页框数
 *return:
 *缺页次数
 */
list<int> lru; //双向链表存放页面，前端为最近使用
unordered_map<int, list<int>::iterator> pos; // 存放页面在链表中的迭代器
int faults = 0;
for (int p : pages) {
    auto it = pos.find(p);
    if (it == pos.end()) {
        // 缺页
        faults++;
        if ((int)lru.size() == capacity) {
            // 内存满，淘汰最久未使用页面
            int old = lru.back();
            pos.erase(old);
            lru.pop_back();
        }
    } else {
        // 命中，将页面移到链表前端
        lru.erase(it->second);
    }
    lru.push_front(p); //将当前页面放入前端，并更新迭代器
    pos[p] = lru.begin();
}
return faults;
}

```

```

int simulateOPT(const vector<int>& pages, int capacity) {
    /*
     *模拟 OPT 最佳页面置换
     *参数:
     *pages:页号序列
     *capacity:页框数
     *return:
     *缺页次数
     */
    vector<int> mem; //内存中页面列表
    int faults = 0;
    int N = pages.size();
}

```

```

for (int i = 0; i < N; ++i) {
    int p = pages[i];

    if (find(mem.begin(), mem.end(), p) != mem.end()) { //判断是否命中
        continue;
    }
    // 缺页
    faults++;
    if ((int)mem.size() < capacity) { // 判断是否未滿，是則加入
        mem.push_back(p);
    } else {
        // 已滿，找未來最晚使用的頁面
        int victim = 0, farthest = i + 1;
        for (int j = 0; j < (int)mem.size(); ++j) {
            int q = mem[j];
            int next_use = N;
            for (int k = i + 1; k < N; ++k) { //查找該頁面下一次使用的位置
                if (pages[k] == q) { next_use = k; break; }
            }
            if (next_use > farthest) {
                farthest = next_use;
                victim = j;
            }
        }
        mem[victim] = p; //用新頁面替換
    }
}
return faults;
}

```

```

int simulateNRU(const vector<int>& pages, int capacity) {
    /*
    *模拟 NRU 非最近使用页面置换
    *参数:
    *pages:页号序列
    *capacity:页框数
    *return:
    *缺页次数
    */
    vector<int> mem; // 内存中页面列表
    vector<bool> refbit; //参考位数组
    int faults = 0;
    for (int p : pages) {

```

```

auto it = find(mem.begin(), mem.end(), p);
if (it != mem.end()) {
    // 判断是否命中，如是，设置参考位为 true
    refbit[it - mem.begin()] = true;
    continue;
}
// 如未命中
faults++;
if ((int)mem.size() < capacity) { // 判断是否未满足，如是，加入页面并设置参考位
    mem.push_back(p);
    refbit.push_back(true);
} else {
    // 如已满，寻找参考位为 false 的页面淘汰
    int victim = -1;
    for (int i = 0; i < (int)mem.size(); ++i) {
        if (!refbit[i]) { victim = i; break; }
    }
    if (victim < 0) {
        // 若所有页面都被访问过，选择第一个作为牺牲页
        victim = 0;
    }
    mem[victim] = p;
    // 替换并重置参考位
    fill(refbit.begin(), refbit.end(), false);
    refbit[victim] = true;
}
}
return faults;
}

```

```

int main() {
    const int N_INSTR = 320; // 指令总数
    const int INSTR_PER_PAGE = 10; // 每页指令数
    auto seq = gen_instruction_sequence(N_INSTR); // 生成指令序列
    auto pages = addr_to_page(seq, INSTR_PER_PAGE); // 转换为页号序列

    cout << fixed << setprecision(2); // 设置输出格式：两位小数
    cout << " 页框    FIFO(%)    LRU(%)    OPT(%)    NRU(%)\\n";
    for (int cap = 4; cap <= 32; ++cap) {
        int f_fifo = simulateFIFO(pages, cap);
        int f_lru = simulateLRU(pages, cap);
        int f_opt = simulateOPT(pages, cap);
        int f_nru = simulateNRU(pages, cap);
    }
}

```



```

//命中率 = (1 - 缺页次数/总指令数) * 100%
double h_fifo = (1.0 - f_fifo / double(N_INSTR)) * 100;
double h_lru  = (1.0 - f_lru  / double(N_INSTR)) * 100;
double h_opt   = (1.0 - f_opt   / double(N_INSTR)) * 100;
double h_nru   = (1.0 - f_nru   / double(N_INSTR)) * 100;

    cout << setw(5) << cap << "      " << setw(6) << h_fifo << "    " << setw(7) << h_lru  <<
"    " << setw(7) << h_opt   << "    " << setw(7) << h_nru   << "\n";
}
return 0;
}

```